Complete High-Availability Web Application Setup - Manual Steps

📋 Table of Contents

🌐 Network Configuration

Step 1: Create VPC Network

Navigate to VPC network → VPC networks and click Create VPC Network.

Configure:

- Name: web-app-network
- Subnet Creation Mode: Custom

Click Add subnet:

- Name: web-app-subnet
- Region: us-central1
- IP stack type: IPv4 (single-stack)
- IPv4 range: 10.1.2.0/24
- Dynamic routing mode: Regional
- Click Create

Step 2: Configure Firewall Rules

Navigate to VPC network → Firewall and click Create Firewall Rule.

Configure:

- Name: allow-http-web-app
- Target tags: web-app
- Source IP ranges: 0.0.0.0/0
- Protocols and ports: TCP → 80
- Click Create

🏥 Health Check Setup

Step 3: Create Health Check

Navigate to Compute Engine → Health Checks and click Create Health Check.

Configure:

- Name: http-health-check
- Protocol: HTTP
- Port: 80

- Request Path: /health
- Check interval: 5 seconds
- Timeout: 5 seconds
- Healthy threshold: 2
- Unhealthy threshold: 2
- Click Create

💻 Instance Configuration

Step 4: Create Instance Template

Navigate to Compute Engine → Instance Templates and click Create Instance Template.

Configure:

- Name: web-app-template
- Machine type: e2-medium
- Boot disk: Click Change → Select Debian 12
- Firewall: Check Allow HTTP traffic

Advanced Options → Networking:

- Network tags: Add web-app

Management → Startup script:

```
#!/bin/bash
apt-get update
apt-get install apache2 -y
service apache2 restart
mkdir -p /var/www/html
echo "<h1>Production Web Server: $(hostname)</h1>" | tee /var/www/html/index.html
echo "Healthy" > /var/www/html/health
```

- Click Create

Step 5: Create Managed Instance Group

Navigate to Compute Engine → Instance Groups and click Create Instance Group.

Configure:

- Name: web-app-mig
- Location: Multiple zones → Select us-central1-a, us-central1-b, us-central1-f
- Instance template: web-app-template
- Autoscaling: Enable
- Min instances: 2
- Max instances: 5
- Target CPU utilization: 70
- Health Check: http-health-check
- Initial delay: 60 seconds
- Click Create

📍 Static IP Configuration

Step 6: Reserve Static IP

Navigate to VPC network → External IP addresses and click Reserve Static Address.

Configure:

- Name: web-app-ip
- Region: us-central1
- Type: Regional
- Attached to: None
- Click Reserve *(Copy the IP address that appears, you will need it later)*

⚖ Load Balancer Setup

Step 7: Create Backend Service and Load Balancer

Navigate to Network Services → Load Balancing and click Create Load Balancer. Under Network Load Balancer (TCP/UDP/SSL) → Click Start Configuration. Select From Internet to my VMs → Click Continue.

Backend Configuration:

- Name: web-app-backend
- Region: us-central1
- Backend type: Instance group
- Instance group: web-app-mig
- Port: 80
- Health check: http-health-check
- Session affinity: Client IP

Frontend Configuration:

- Name: web-app-frontend
- IP address: Select web-app-ip
- Port: 80
- Click Create

Step 8: Enable Access Logs

- Navigate to Network Services → Load Balancing
- Click on your load balancer (web-app-backend)
- In the Backend section, click the pencil icon
- Check Enable Logging
- Set Sample rate to 1.0
- Click Save

📊 Monitoring & Alerting

Step 9: Create Alert Policies

Navigate to Monitoring → Alerting and click Create Policy.

Alert 1: High CPU Usage

- Add Condition:
  - Resource type: GCE VM Instance

- Metric: instance/cpu/utilization
  - Filter: instance_group = "web-app-mig"
  - Condition: > 80% for 1 minute
- Configure:
  - Name: High CPU Alert - Web App
  - Severity: Warning
  - Notifications: Add your email
- Click Save

Alert 2: HTTP 5xx Errors

- Create New Alert Policy
- Add Condition:
  - Resource type: HTTP(S) Load Balancer
  - Metric: loadbalancing.googleapis.com/https/request_count
  - Filter: response_code_class = 5xx and backend_name = "web-app-backend"
  - Condition: > 5% of total requests for 2 minutes
- Configure:
  - Name: High 5xx Errors - Web App
  - Severity: Critical
- Click Save

Alert 3: Unhealthy Instances

- Create New Alert Policy
- Add Condition:
  - Resource type: GCE Instance Group
  - Metric: instance_group/unhealthy_instances
  - Condition: > 1 for 2 minutes
- Configure:
  - Name: Unhealthy Instances Detected
  - Severity: Warning
- Click Save

Step 10: Create Monitoring Dashboard

Navigate to Monitoring → Dashboards and click Create Dashboard. Name it Web App Production Dashboard.

Add Widgets:

- Widget 1: CPU Utilization
  - Type: Line Chart
  - Metric: instance/cpu/utilization
  - Filter: instance_group = "web-app-mig"
  - Aggregation: Mean over 1 minute
  - Title: CPU Usage - All Instances
- Widget 2: Instance Count
  - Chart Type: Stacked Bar
  - Metric: instance_group/current_size
  - Filter: instance_group_name = "web-app-mig"
  - Title: Running Instances (Min: 2, Max: 5)
- Widget 3: Health Check Status

- Chart Type: Single Stat
    - Metric: loadbalancing.googleapis.com/https/backend_healthy
    - Title: Healthy Backend Instances
- Widget 4: Request Count
    - Chart Type: Line Chart
    - Metric: loadbalancing.googleapis.com/https/request_count
    - Filter: backend_name = "web-app-backend"
    - Aggregation: Sum over 1 minute
    - Title: Total Requests (qps)
- Widget 5: Response Codes
    - Chart Type: Stacked Bar
    - Metric: loadbalancing.googleapis.com/https/request_count
    - Group By: response_code_class
    - Title: HTTP Response Codes (2xx, 4xx, 5xx)
- Widget 6: Latency (P99)
    - Chart Type: Line Chart
    - Metric: loadbalancing.googleapis.com/https/backend_latencies
    - Filter: backend_name = "web-app-backend"
    - Aggregation: p99
    - Title: Backend Latency - P99

Step 11: Setup Uptime Checks

Navigate to Monitoring → Uptime Checks and click Create Uptime Check.

Configure:

- Name: Web App Production
- Protocol: HTTP
- Hostname: Enter your static IP
- Path: /health
- Check Frequency: 1 minute
- Regions: Select at least 3 (US, Europe, Asia)
- Enable alert on failure for 2 minutes
- Click Create

Step 12: Configure Notification Channels

Navigate to Monitoring → Alerting → Notification Channels and click Create Channel.

- Email Setup: Select Email and enter email addresses.
- Slack Setup (Optional): Select Webhook and paste Slack webhook URL.
- Click Save

✅ Verification

Test the Load Balancer

- Open browser
- Navigate to http://YOUR_STATIC_IP

- Verify web server message appears
- Refresh multiple times to see hostname change

Test Auto-Healing

- Go to Compute Engine → VM Instances
- Stop or delete one VM
- Wait for MIG to detect failure
- Verify new instance is created automatically

View Logs

- Navigate to Logging → Logs Explorer
- Use filter: resource.type="http_load_balancer"
- Click Run Query

Test Uptime Check

- Navigate to Monitoring → Uptime Checks
- Click on your uptime check
- Verify successful check status

🔧 Troubleshooting Commands

SSH into Instance

```
gcloud compute ssh INSTANCE_NAME --zone=ZONE
```

Check Apache Status

```
sudo systemctl status apache2
```

View Startup Script Logs

```
sudo journalctl -u google-startup-scripts.service -f
```

Test Health Endpoint

```
curl http://localhost/health
```

📝 Important Notes

- Firewall Tags: Ensure instances have web-app tag
- Health Check Path: Must return 200 OK for /health
- Session Affinity: Uses Client IP for stickiness
- Auto-Scaling: Adjusts between 2-5 instances based on CPU
- Logging: All requests logged with 1.0 sample rate